

概述

本文介绍了使用 LIN_Slave_node 前，Flash 中 ID 以及 NAD 初始化事宜。以及进行了基础通讯演示。

目录

1	概述	7
2	基础通讯演示	7
3	ID 以及 NAD 加载（可选）	8
3.1	ID 以及 NAD 加载的两条路径	8
3.2	ID 以及 NAD 的存储	9
3.3	开启从 Flash 加载 ID 以及 NAD	11
4	总线空闲自动睡眠（可选）	12
5	LIN 睡眠唤醒	13
6	单帧传输	14
6.1	单帧传输接口	14
6.2	单帧传输演示	16
7	多帧传输	17
7.1	多帧传输结构	17
7.2	多帧传输接口	17
7.3	多帧传输演示	19
7.4	新增多帧传输服务	19
8	传输超时	21
8.1	N_As 超时	21
8.2	N_Cr 超时	22
8.3	N_As 或 N_Cr 超时后置起 flag	23
8.4	N_As 超时后通过 FF, CF 帧恢复	24
8.5	N_As 超时后通过 SF 帧恢复	25
8.6	N_Cr 超时后通过 FF, CF 帧恢复	26
8.7	N_Cr 超时后通过 SF 帧恢复	27

图片列表

图 1-1: 目录结构	7
图 2-1: 基础通信	7
图 3-1: ID NAD 加载流程	8
图 6-1: 主收从发通信	16
图 7-1: 传输层	17
图 7-2: 0x22, 0x2E 通信	19
图 7-3: 0x10 通信	20
图 8-1: N_As 超时 (单帧)	21
图 8-2: N_As 超时截图	21
图 8-3: N_Cr 超时 (多帧)	22
图 8-4: N_Cr 超时截图	22
图 8-5: 恢复正常通信	24
图 8-6: 恢复正常通信	25
图 8-7: 恢复正常通信	26
图 8-8: 恢复正常通信	27

表格列表

SPIN TROL

版本历史

版本	日期	作者	状态	变更
A/0	2023-7-30	HangSu	Outdated	首次发布。
A/1	2023-9-1	HangSu	Outdated	1. 更新 章节 3 2. 更新 章节 4 3. 更新 章节 5
A/2	2023-9-21	HangSu	Outdated	1. 更新 章节 3 2. 更新 章节 4 3. 更新 章节 5 4. 更新 章节 7 5. 更新 章节 8
A/3	2023-10-11	HangSu	Outdated	更新 章节 3.1
A/4	2023-12-29	HangSu	Outdated	1. 增加 章节 1 2. 增加 章节 3.2 3. 增加 章节 6 4. 更新 章节 7 5. 更新 章节 8
A/5	2024-1-4	HangSu	Released	1. 更新 章节 1 2. 更新 章节 3

术语或缩写

术语或缩写	描述

SPIN TROL

1 概述

LIN_Slave_node 下有 Multi 和 Single 两个工程，Multi 同时支持多帧以及单帧通信，占用 RAM 资源较多；Single 只支持单帧通信，占用 RAM 资源相对较少。

Multi.uvprojx 对应 multi 下的 lin_cfg.c\h；Single.uvprojx 对应 single 下的 lin_cfg.c\h，也就是 multi 下的 lin_cfg.c\h 与 single 下的 lin_cfg.c\h 不通用。

图 1-1：目录结构

```
LIN_Slave_node --> MDK-ARM --> Multi.uvprojx
                        --> Single.uvprojx
--> multi --> lin_cfg.c
                --> lin_cfg.h
--> Node configuration files --> LINDiagnostic.ldf
--> single --> lin_cfg.c
                --> lin_cfg.h
--> isr.c
--> isr.h
--> main.c
```

注意：目前软件只支持 Slave LIN2.1。

2 基础通讯演示

主机发 0x3C 帧头，之后发 0x11， 0x06， 0xB2， 0x00， 0x21， 0x00， 0x01， 0x00，并按经典校验发送 checksum。

之后主机发 0x3D 帧头，从机将准备好的数据回给主机，并按经典校验发送 checksum。

图 2-1：基础通信

时间	通道	ID(hex)	名称	事件类型	方向	长度	数据(hex)	解释
314.328023	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.346502	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.365530	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.381477	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.398774	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.414479	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.432661	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.449901	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.467500	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.483501	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.501249	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.518561	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.536297	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.553644	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.573502	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.592536	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.610437	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
314.626680	LIN 1	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
314.644436	LIN 1	3D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	

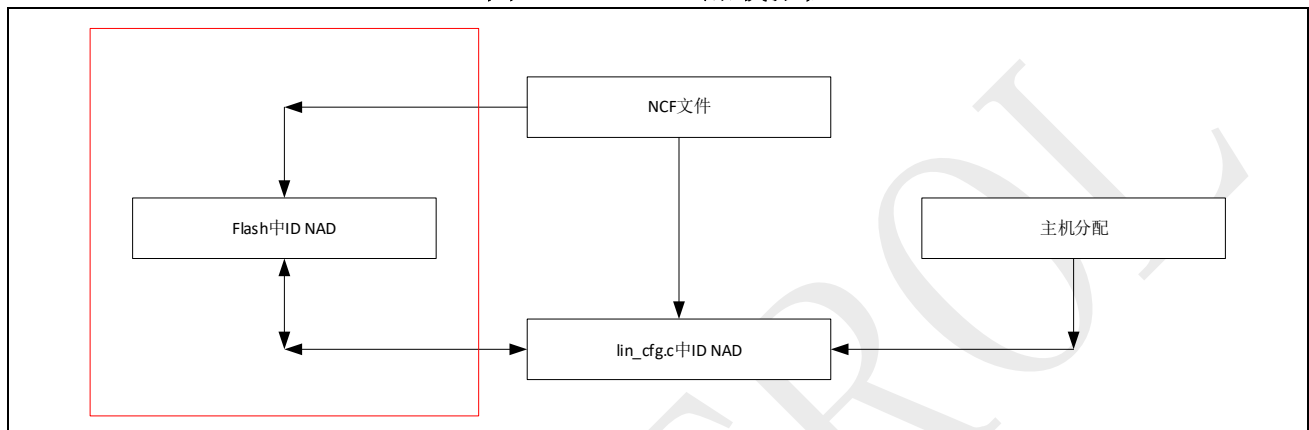
显示错误视图 Rx计数: 13 Tx计数: 13 错误计数: 0

3 ID 以及 NAD 加载（可选）

3.1 ID 以及 NAD 加载的两条路径

ID 以及 NAD 的加载有两条路径，如图 3-1 所示。一路是通过 NCF 生成 lin_cfg.c 中的 lin_configuration_RAM 与 lin_configured_NAD，一路是通过 Flash 中 ID 以及 NAD 更新 lin_configuration_RAM 与 lin_configured_NAD（默认关闭）。

图 3-1: ID NAD 加载流程



通过 Flash 中 ID 以及 NAD 更新 lin_configuration_RAM 与 lin_configured_NAD 的代码如下所示：

lin_llid_uart.c

```
void lin_init_id_or_nad_from_flash ()
{
    uint32_t i;

    /* Get data from Flash */
    PHWLIB->FLASHC_Read ((uint32_t *) lin_configuration_ID_and_NAD,
        OPERATION_ADDR, (((LIN_SIZE_OF_CFG + 1) / 8 + 1) * 2));
    /* If data format is right, use the data */
    if ((lin_configuration_ID_and_NAD[0] == 0x00) &&
        (lin_configuration_ID_and_NAD[LIN_SIZE_OF_CFG - 1] == 0xFF))
    {
        for (i = 0; i < LIN_SIZE_OF_CFG; i++)
        {
            lin_configuration_RAM[i] = lin_configuration_ID_and_NAD[i];
            printf ("%x\n", lin_configuration_RAM[i]);
        }

        lin_configured_NAD = lin_configuration_ID_and_NAD[LIN_SIZE_OF_CFG];
        printf ("%x\n", lin_configured_NAD);
    }
    /* If data format is not right, init Flash, do not change data in
    lin_configuration_RAM and lin_configured_NAD */
    else
    {
        for (i = 0; i < LIN_SIZE_OF_CFG; i++)
        {
            lin_configuration_ID_and_NAD[i] = lin_configuration_ROM[i];
        }
        lin_configuration_ID_and_NAD[LIN_SIZE_OF_CFG] = lin_initial_NAD;
    }
}
```



```

pHWLIB->FLASHC_EraseSector (OPERATION_ADDR) ;
pHWLIB->FLASHC_Program ((uint32_t *) lin_configuration_ID_and_NAD,
OPERATION_ADDR, (((LIN_SIZE_OF_CFG + 1) / 8 + 1) * 2)) ;
printf ("Program\n") ;

for (i =0; i < LIN_SIZE_OF_CFG; i++)
{
    printf ("%x\n", lin_configuration_RAM[i]) ;
}

printf ("%x\n", lin_configured_NAD) ;
}
}

```

- 1) 读取 Flash 0x1000F000 处数据，并判断数据是否合法，例如读取数据为 0x00, 0x01, 0x06, 0x03, 0x08, 0x04, 0x0B, 0x3C, 0x3D, 0xFF, 0x11，其中 0x00 为 ID 起始标志位，0x01, 0x06, 0x03, 0x08, 0x04, 0x0B, 0x3C, 0x3D 为 ID，0xFF 为 ID 结束标志位，0x11 为初始 NAD；
- 2) 该数据合法，用 Flash 中读到的数据更新 lin_configuration_RAM 与 lin_configured_NAD；
- 3) 如果数据不合法，例如全为 0xFF，就用 lin_configuration_ROM 与 lin_initial_NAD 更新 Flash 中数据，维持 lin_configuration_RAM 与 lin_configured_NAD 不变；

开启从 Flash 加载 ID 以及 NAD 的操作会占用一段 Flash 空间，如果不需要，可将其关闭。

注意：	数据合法判定依据仅和格式有关，如果调试过程重新定义了 Idf 文件中的 ID，NAD 或 lin_cfg 中的 ID，NAD，需要采用全擦方式下载，否则程序会用 Flash 0x1000F000 处的旧数据。
-----	--

3.2 ID 以及 NAD 的存储

从机收到主机 0x3C（诊断帧）对应的 B6（存储）服务时，会将当前 NAD 以及 ID 存储到 Flash 中，该过程会调用 pHWLIB->FLASHC_EraseSector，以及 pHWLIB->FLASHC_Program，该函数执行过程均会关闭 Flash XIP，期间任何中断执行均失败（Bus error）。

解决方法任选其一：

- 方法一：在调用 pHWLIB->FLASHC_EraseSector，以及 pHWLIB->FLASHC_Program 前关闭中断响应，在调用 pHWLIB->FLASHC_EraseSector，以及 pHWLIB->FLASHC_Program 后重新开启中断响应；
- 方法二：将中断向量表，以及所有中断函数放到 RAM 中，参见例程《18_5_Flash_With_ISR_In_RAM》。

lin_lld_uart.c（方法一）

```

void lin_init_id_or_nad_from_flash()
{
    uint32_t i;

    /* Get data from Flash */

```

```

    pHWLIB->FLASHC_Read((uint32_t *)lin_configuration_ID_and_NAD,
OPERATION_ADDR, (((LIN_SIZE_OF_CFG + 1)/8 + 1) *2));
    /* If data format is right, use the data */
    if ((lin_configuration_ID_and_NAD[0] == 0x00) &&
(lin_configuration_ID_and_NAD[LIN_SIZE_OF_CFG - 1] == 0xFF))
    {
        for (i = 0; i < LIN_SIZE_OF_CFG; i++)
        {
            lin_configuration_RAM[i] = lin_configuration_ID_and_NAD[i];
            printf("%x\n", lin_configuration_RAM[i]);
        }

        lin_configured_NAD = lin_configuration_ID_and_NAD[LIN_SIZE_OF_CFG];
        printf("%x\n", lin_configured_NAD);
    }
    /* If data format is not right, init Flash, do not change data in
lin_configuration_RAM and lin_configured_NAD */
    else
    {
        for (i = 0; i < LIN_SIZE_OF_CFG; i++)
        {
            lin_configuration_ID_and_NAD[i] = lin_configuration_ROM[i];
        }
        lin_configuration_ID_and_NAD[LIN_SIZE_OF_CFG] = lin_initial_NAD;

        __disable_irq();
        pHWLIB->FLASHC_EraseSector(OPERATION_ADDR);
        pHWLIB->FLASHC_Program((uint32_t *)lin_configuration_ID_and_NAD,
OPERATION_ADDR, (((LIN_SIZE_OF_CFG + 1)/8 + 1) *2));
        __enable_irq();

        printf("Program\n");

        for (i = 0; i < LIN_SIZE_OF_CFG; i++)
        {
            printf("%x\n", lin_configuration_RAM[i]);
        }

        printf("%x\n", lin_configured_NAD);
    }
}

void lin_save_configuration()
{
    uint32_t i;
    if (lin_save_configuration_flg == 1)
    {
        for (i = 0; i < LIN_SIZE_OF_CFG; i++)
        {
            lin_configuration_ID_and_NAD[i] = lin_configuration_RAM[i];
        }
        lin_configuration_ID_and_NAD[LIN_SIZE_OF_CFG] = lin_configured_NAD;

        __disable_irq();
        pHWLIB->FLASHC_EraseSector(OPERATION_ADDR);
        pHWLIB->FLASHC_Program((uint32_t *)lin_configuration_ID_and_NAD,
OPERATION_ADDR, (((LIN_SIZE_OF_CFG + 1)/8 + 1) *2));
        __enable_irq();

        lin_save_configuration_flg = 0;
    }
}

```

```
}
```

3.3 开启从 Flash 加载 ID 以及 NAD

如果需要开启从 Flash 加载 ID 以及 NAD, 可在 lin.h 中定义以下宏, 其中 OPERATION_ADDR 代表 ID 以及 NAD 在 Flash 中存储地址。

lin.h

```
#define OPERATION_ADDR      0x1000F000  
#define LOAD_ID_NAD_FROM_FLASH
```

4 总线空闲自动睡眠（可选）

为了便于调试，默认总线空闲自动睡眠功能是关闭的，否则，当 LIN 总线空闲一段时间后，LIN_Slave_node 会自动进入睡眠状态。

如需开启，可以定义宏 BUS_IDLE_AUTO_SLEEP，其作用代码如下所示：

lin_lld_uart.c

```

LIN_Init (UART1, LIN_SLAVE, LIN_BAUD_RATE) ;

LIN_SetIDFilter (UART1, 0x00, 0x00) ;

UART_SetBusIdleThreshold (UART1, (4*SysInfo.u32SYSCLK)) ;
UART_RunBusIdleCounter (UART1) ;

UART_EnableInt (UART1, UART_INT_LIN_ID_MATCH) ;

/* Enable UART1_IRQn trigger INT in MCU side */
NVIC_EnableIRQ (UART1_IRQn) ;

/* Set the default u32Th, enable the interrupt when FIFO level > u32Th */
UART_SetRxFIFOThreshold (UART1, 8) ;
UART_EnableInt (UART1, UART_INT_RX_TIMEOUT) ;
UART_EnableInt (UART1, UART_INT_RX_FRAME_ERROR) ;
UART_EnableInt (UART1, UART_INT_LIN_BIT_ERROR) ;

#ifdef BUS_IDLE_AUTO_SLEEP
    UART_EnableInt (UART1, UART_INT_LIN_BUS_IDLE) ;
#else
#endif

```

其中阈值时间 UART_SetBusIdleThreshold (UART1, (4*SysInfo.u32SYSCLK)) 默认为 4s，可自行修改。

睡眠操作位于 lin_lld_uart.c 文件 UART1_IRQHandler 中，如下所示。

lin_lld_uart.c

```

else if (UART_GetIntFlag (UART1, UART_INT_LIN_BUS_IDLE) != 0)
{
    printf ("BUS_IDLE\n") ;
    lin_lld_uart_set_low_power_mode () ;

    UART_ClearInt (UART1, UART_INT_LIN_BUS_IDLE) ;
}

```

5 LIN 睡眠唤醒

除了总线空闲时自动睡眠，还可以在收到睡眠指令后，进入 `lin_common_proto.c`，调用 `lin_llid_set_low_power_mode()` 实现芯片睡眠。

`lin_common_proto.c`

```
void lin_update_rx_diag_frame
(
)
{
    /* Check goto sleep */
    if (0x00 == lin_llid_response_buffer[1])
    {
        lin_goto_sleep_flg = 1;
        lin_llid_set_low_power_mode();
        return;
    }
    /* TL support */
    /* Single frame support */
    #if (_TL_FRAME_SUPPORT_ == _TL_SINGLE_FRAME_)
        /* Copy response to diagnostic PDU in TL */
        lin_tl_put_pdu();
    #endif /* End (_TL_FRAME_SUPPORT_ == _TL_SINGLE_FRAME_) */
    /* Process PDU income */
    lin_tl_handler();
}
```

将 LIN 管脚接地，并维持低电平大于 80us，将会唤醒电源管理模块。

6 单帧传输

6.1 单帧传输接口

lin_pFrameBuf[]位于 lin_cfg.c 中，Frames 个数和长度均和 ldf 文件对应。

lin_cfg.c

```
/* definition and initialization of signal array */
l_u8  lin_pFrameBuf[LIN_FRAME_BUF_SIZE] =
{

    0xf0 /* 0 : 11110000 */ /* start of frame LI0_FrontLeftDoorMessage */

    ,0x00 /* 1 : 00000000 */ /* start of frame LI0_MasterToFrontControl */
    ,0xff /* 2 : 11111111 */

    ,0xfe /* 3 : 11111110 */ /* start of frame LI0_FrontLeftDoorErrEvent */

    ,0xf0 /* 4 : 11110000 */ /* start of frame LI0_FrontToMasterMessage */

    ,0xc4 /* 5 : 11000100 */ /* start of frame LI0_FrontToMasterEventMessage */
    ,0xf0 /* 6 : 11110000 */

};
```

lin_pFrameBuf[]中 FrontLeftDoorMessage, FrontLeftDoorErrEvent, FrontToMasterMessage, FrontToMasterEventMessage 是从机发送给主机的数据(见 LINDiagnostic.Ldf 文件 originator 列), 修改 lin_pFrameBuf[]中对应数据, 可在对应帧头来临时, 自动将数据发送出去。

LINDiagnostic.Ldf

```
Frames // Unconditional Frames Definition
{
    FrontLeftDoorMessage: 1, FrontLeftDoor, 1 //frame name, frame ID(decimal),
    originator, frame length(byte)
    {
        FrontLeftDoorSignal, 0; // signal name, signal offset in frame(bit)
    }
    //error frame will use eventrigger frame
    FrontLeftDoorErrEvent: 3, FrontLeftDoor, 1
    {
        FrontLeftDoorErrSig, 0;
    }

    MasterToFrontControl: 6, LINMaster, 2{
        MasterToFront, 0;
    }

    FrontToMasterMessage: 8, FrontLeftDoor, 1{
        FrontToMasterSignal, 0;
    }
    FrontToMasterEventMessage: 4, FrontLeftDoor, 2{
```

```
FrontToMasterEventSignal, 8; // signal name, signal offset in
frame(bit): because this frame is relevant to event triggered frame, offset
must >= 8
}
} // End of Unconditional Frames Definition
```

接收状态下，在校验通过后，可以通过 response_buffer[] 获取主机发来的数据，response_buffer[0]对应接收长度，response_buffer[x](0 < x < (response_buffer[0] + 1))对应接收到的实际数据。

lin_llc_uart.c

```
else if (UART_GetIntFlag(UART1, UART_INT_RX_REQ) != 0)
{
    ptr++;
    for (i = 0; i < (response_buffer[0] + 1); i++)
    {
        *(ptr) = UART_ReadByte(UART1);
        ptr++;
        cnt_byte++;
    }

    /* Check bytes received fully */
    if (cnt_byte == (response_buffer[0] + 1))
    {
        /* checksum checking */
        if (lin_checksum(response_buffer, pid) ==
response_buffer[(response_buffer[0]+1)])
        {
            /******
            /** RX Buffer Full - Checksum OK      */
            /******

            /* Insert logic operation */

            /* set lin status: successful_transfer */
            l_status.byte |= LIN_STA_SUCC_TRANSFER;

            /* trigger callback */
            CALLBACK_HANDLER((l_ifc_handle)ifc, LIN_LLD_RX_COMPLETED,
current_id);
        }
        else
        {
            /******
            /** RX Buffer Full - Checksum ERROR    */
            /******

            /* set lin status: error_in_response, checksum_error */
            l_status.byte |= (LIN_STA_ERROR_RESP|LIN_STA_CHECKSUM_ERR);
            /* trigger callback */
            CALLBACK_HANDLER((l_ifc_handle)ifc, LIN_LLD_CHECKSUM_ERR,
current_id);
        }
        lin_goto_idle_state();
    }
}
```

6.2 单帧传输演示

主机分别发 0x01, 0x03, 0x08, 0x04 的帧头, 从机将 lin_pFrameBuf[]对应位置的数据返回给主机。

图 6-1: 主收从发通信

时间	通道	ID(hex)	PID(h...	名称	事件类型	方向	长度	数据(hex)
					全部	全部		
12120.141620	LIN 1	01	C1		LIN帧	Rx	1	F0
12120.151417	LIN 1	03	03		LIN帧	Rx	1	FE
12120.161327	LIN 1	08	08		LIN帧	Rx	1	F0
12120.171500	LIN 1	04	C4		LIN帧	Rx	2	C4 F0

7 多帧传输

7.1 多帧传输结构

LIN 除了无条件帧，事件帧这种典型单帧传输，还可以通过传输层，实现应用层和诊断帧（0x3C，0x3D）之间的多帧传输。

多帧传输的过程在传输层中进行，其负责将诊断帧（0x3C）收到的单帧数据组合成一串数据，供应用层使用；或将应用层发出的一串数据分解成单帧数据，供诊断帧（0x3D）发出。

图 7-1：传输层



0x3C 可以分为 FF（首帧）+CF（续帧），或 SF（单帧）；0x3D 同样可以分为 FF（首帧）+CF（续帧），或 SF（单帧），共四种组合。

7.2 多帧传输接口

当主机按照 LIN 协议发送完 3C_FF 以及 3C_CF 或 3C_SF 后，从机在 lin_commontl_proto.c 的 lin_tl_attach_service() 中通过 SID 判断是否为 SERVICE_READ_DATA_BY_IDENTIFY（对应 0x22）服务，是则进入 lin_tl_make_multi_slaveres_pdu（）；

同理，通过 SID 判断是否为 SERVICE_WRITE_DATA_BY_IDENTIFY（对应 0x2E）服务，是则进入 lin_tl_make_multi_slaveres_pdu（）；

lin_tl_attach_service（）

```
case SERVICE_READ_DATA_BY_IDENTIFY:
    /* Response to master - RSID */
    lin_tl_make_multi_slaveres_pdu (SERVICE_READ_DATA_BY_IDENTIFY) ;
    break;
case SERVICE_WRITE_DATA_BY_IDENTIFY:
    /* Response to master - RSID */
    lin_tl_make_multi_slaveres_pdu (SERVICE_WRITE_DATA_BY_IDENTIFY) ;
    break;
```

在 lin_tl_make_multi_slaveres_pdu（）中，通过 Id_receive_message（）解析一共收到的数据及其长度（可通过 recdata[100]修改最大可接收长度），之后设定要响应的数据及其长度（可通过 senddata[10]修改最大可发送长度），最后通过 Id_send_message 放入传输队列。

lin_tl_make_multi_slaveres_pdu ()

```
void lin_tl_make_multi_slaveres_pdu
(
    /* [IN] service identifier */
    l_u8 sid
)
{
    l_u16 length;

    l_u8 recdata[100];
    l_u8 senddata[10] = {0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xaa};

    /* get pdu from rx queue */
    ld_receive_message (&length, recdata) ;

    if (sid == SERVICE_READ_DATA_BY_IDENTIFY)
    {
        // Set response length and senddata according to recdata
        senddata[0] = sid + 0x40;
        length = 10;
    }
    else if (sid == SERVICE_WRITE_DATA_BY_IDENTIFY)
    {
        // Set response length and senddata according to recdata
        senddata[0] = sid + 0x40;
        length = 10;
    }

    ld_send_message (length, senddata) ;
}
```

7.3 多帧传输演示

示例并没有考虑 ISO14229-1, 只是演示 0x22, 0x2E 多帧通讯功能, 分别演示 3C_FF, 3C_CF, 3D_FF, 3D_CF 以及 3C_SF, 3D_FF, 3D_CF 四种组合, 如果减小 Id_send_message () 传入参数 length 大小, 还可以实现 3C_FF, 3C_CF, 3D_SF 以及 3C_SF, 3D_SF 的响应。

图 7-2: 0x22, 0x2E 通信

时间	通道	ID(hex)	PID(h...	名称	事件类型	方向	长度	数据(hex)
					全部	全部		
10111.552585	LIN 1	3C	3C		LIN帧	Tx	8	11 10 0F 22 FF FF FF FF
10111.562365	LIN 1	3C	3C		LIN帧	Tx	8	11 21 FF FF FF FF FF FF
10111.573054	LIN 1	3C	3C		LIN帧	Tx	8	11 22 FF FF FF FF FF FF
10111.582565	LIN 1	3D	7D		LIN帧	Rx	8	11 10 0A 62 FF FF FF FF
10111.592220	LIN 1	3D	7D		LIN帧	Rx	8	11 21 FF FF FF FF AA FF
10123.784459	LIN 1	3C	3C		LIN帧	Tx	8	11 10 0F 2E FF FF FF FF
10123.795501	LIN 1	3C	3C		LIN帧	Tx	8	11 21 FF FF FF FF FF FF
10123.806072	LIN 1	3C	3C		LIN帧	Tx	8	11 22 FF FF FF FF FF FF
10123.815922	LIN 1	3D	7D		LIN帧	Rx	8	11 10 0A 6E FF FF FF FF
10123.825922	LIN 1	3D	7D		LIN帧	Rx	8	11 21 FF FF FF FF AA FF
10417.331043	LIN 1	3C	3C		LIN帧	Tx	8	11 06 22 FF FF FF FF FF
10417.341136	LIN 1	3D	7D		LIN帧	Rx	8	11 10 0A 62 FF FF FF FF
10417.350804	LIN 1	3D	7D		LIN帧	Rx	8	11 21 FF FF FF FF AA FF
10433.636612	LIN 1	3C	3C		LIN帧	Tx	8	11 06 2E FF FF FF FF FF
10433.646144	LIN 1	3D	7D		LIN帧	Rx	8	11 10 0A 6E FF FF FF FF
10433.655879	LIN 1	3D	7D		LIN帧	Rx	8	11 21 FF FF FF FF AA FF

7.4 新增多帧传输服务

如果要增加 SID 服务 0x10, 需要在 lin_cfg.c 的 lin_diag_services_supported 中增加定义 0x10, 和 lin_diag_services_flag 中增加定义 0, 并修改 _DIAG_NUMBER_OF_SERVICES_ 大小到 9:

lin_cfg.c

```
const 1_u8 lin_diag_services_supported[_DIAG_NUMBER_OF_SERVICES_] = {0xB2,
0x2E, 0xB7, 0xB0, 0xB3, 0xB6, 0xB1, 0x22, 0x10};
1_u8 lin_diag_services_flag[_DIAG_NUMBER_OF_SERVICES_] = {0, 0, 0, 0, 0, 0, 0, 0,
0, 0};
```

lin_cfg.h

```
#define _DIAG_NUMBER_OF_SERVICES_ 9
```

在 lin_tl_attach_service () 添加处理逻辑

lin_tl_attach_service ()

```
case SERVICE_READ_DATA_BY_IDENTIFY:
/* Response to master - RSID */
lin_tl_make_multi_slaveres_pdu (SERVICE_READ_DATA_BY_IDENTIFY) ;
break;
case SERVICE_WRITE_DATA_BY_IDENTIFY:
/* Response to master - RSID */
lin_tl_make_multi_slaveres_pdu (SERVICE_WRITE_DATA_BY_IDENTIFY) ;
```

```
break;
case SERVICE_SESSION_CONTROL:
/* Response to master - RSID */
lin_tl_make_multi_slaveres_pdu (SERVICE_SESSION_CONTROL) ;
break;
```

在 lin_tl_make_multi_slaveres_pdu () 添加处理逻辑

lin_tl_make_multi_slaveres_pdu ()

```
void lin_tl_make_multi_slaveres_pdu
(
/* [IN] service identifier */
l_u8 sid
)
{
l_u16 length;

l_u8 recdata[100];
l_u8 senddata[10] = {0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xaa};

/* get pdu from rx queue */
ld_receive_message (&length, recdata) ;

if (sid == SERVICE_READ_DATA_BY_IDENTIFY)
{
// Set response length and senddata according to recdata
senddata[0] = sid + 0x40;
length = 10;
}
else if (sid == SERVICE_WRITE_DATA_BY_IDENTIFY)
{
// Set response length and senddata according to recdata
senddata[0] = sid + 0x40;
length = 10;
}
else if (sid == SERVICE_SESSION_CONTROL)
{
// Set response length and senddata according to recdata
senddata[0] = sid + 0x40;
length = 10;
}

ld_send_message (length, senddata) ;
}
```

重新编译下载，可实现 0x10 通信。

图 7-3: 0x10 通信

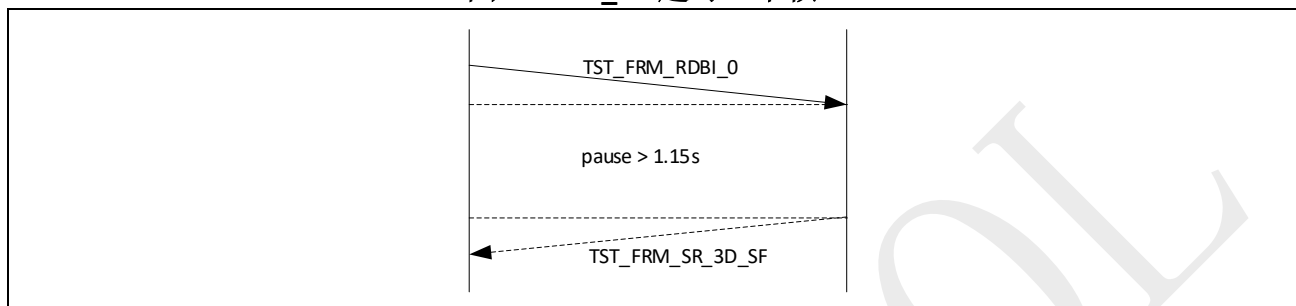
14052.637222	LIN 1	3C	3C	...	Tx	8	11 10 0F 10 FF FF FF FF
14052.646505	LIN 1	3C	3C	LIN响应	Tx	8	11 21 FF FF FF FF FF FF
14052.655618	LIN 1	3C	3C	LIN响应	Tx	8	11 22 FF FF FF FF FF FF
14052.664905	LIN 1	3D	7D	LIN响应	Rx	8	11 10 0A 50 FF FF FF FF
14052.674918	LIN 1	3D	7D	LIN响应	Rx	8	11 21 FF FF FF FF AA FF
14058.907043	LIN 1	3C	3C	LIN响应	Tx	8	11 06 10 FF FF FF FF FF
14058.917088	LIN 1	3D	7D	LIN响应	Rx	8	11 10 0A 50 FF FF FF FF
14058.927001	LIN 1	3D	7D	LIN响应	Rx	8	11 21 FF FF FF FF AA FF

8 传输超时

8.1 N_As 超时

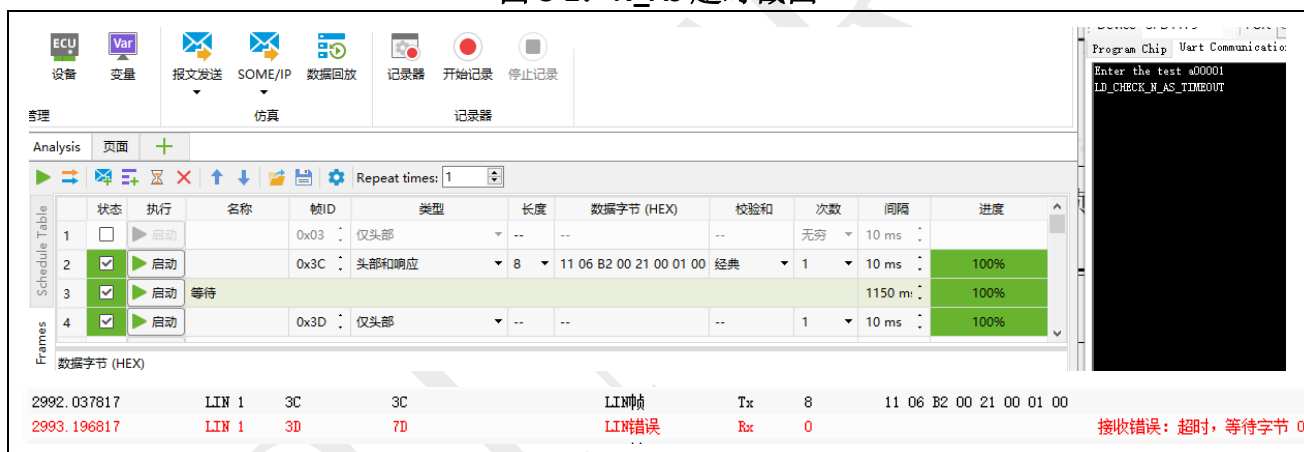
正常通讯 0x3C 帧（主发从收）和 0x3D 帧（主收从发）间隔不超 1.15s，超过 1.15s 则会触发 N As 超时，如图 8-1 所示。

图 8-1: N_As 超时 (单帧)



超时实验截图如图 8-2 所示。

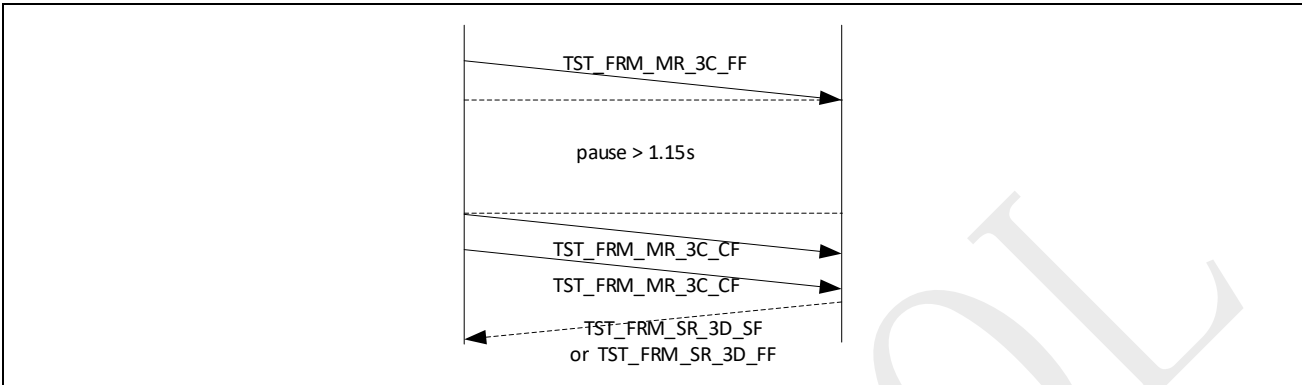
图 8-2: N As 超时截图



8.2 N_Cr 超时

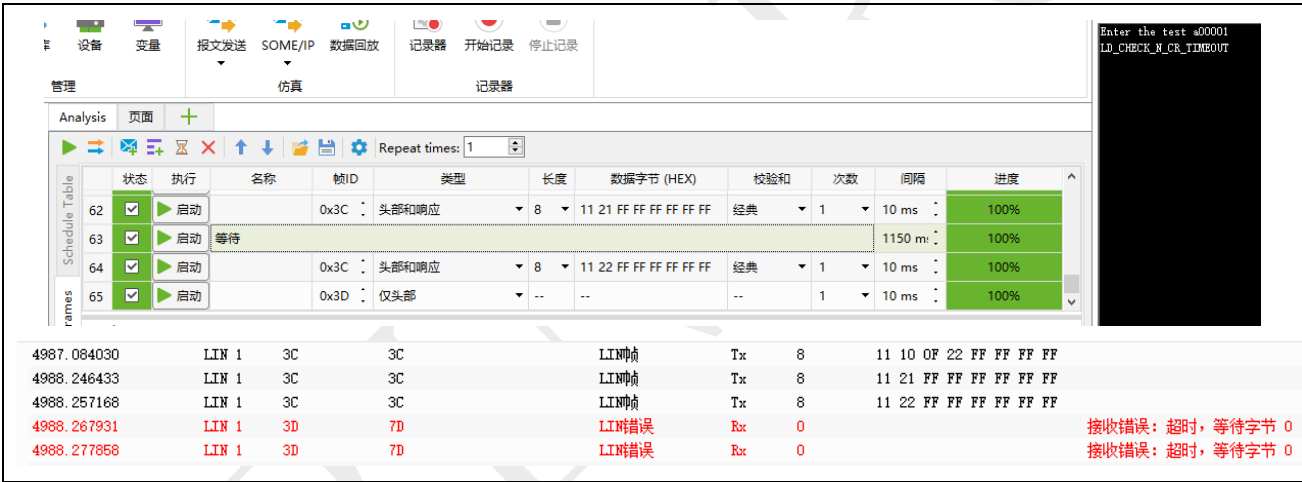
正常通讯 0x3C 帧（FF）和 0x3C 帧（CF）间隔不超 1.15s，超过 1.15s 则会触发 N_Cr 超时，如图 8-3 所示。

图 8-3：N_Cr 超时（多帧）



超时实验截图如图 8-4 所示。

图 8-4：N_Cr 超时截图



8.3 N_As 或 N_Cr 超时而后置起 flag

N_As, N_Cr 超时会进入 lin_lld_uart.c 中 lin_Cr_or_As_timeout, 代码如下:

lin_lld_uart.c

```
void lin_Cr_or_As_timeout ()
{
    if (LD_CHECK_N_CR_TIMEOUT == tl_check_timeout_type)
    {
        if (0 == --tl_check_timeout)
        {
            printf ("LD_CHECK_N_CR_TIMEOUT\n");
            /* update status of transport layer */
            tl_service_status = LD_SERVICE_ERROR;
            tl_receive_msg_status = LD_N_CR_TIMEOUT;
            tl_rx_msg_status = LD_N_CR_TIMEOUT;
            tl_check_timeout_type = LD_NO_CHECK_TIMEOUT;
            tl_diag_state = LD_DIAG_IDLE;
        }
    }

    if (LD_CHECK_N_AS_TIMEOUT == tl_check_timeout_type)
    {
        if (0 == --tl_check_timeout)
        {
            printf ("LD_CHECK_N_AS_TIMEOUT\n");
            /* update status of transport layer */
            tl_service_status = LD_SERVICE_ERROR;
            tl_tx_msg_status = LD_N_AS_TIMEOUT;
            tl_check_timeout_type = LD_NO_CHECK_TIMEOUT;
            tl_diag_state = LD_DIAG_IDLE;
        }
    }
}
```

8.4 N_As 超时后通过 FF, CF 帧恢复

N_As 超时事件发生后, 可通过 FF, CF 帧恢复正常通信, 如图 8-5 所示。

图 8-5: 恢复正常通信

通道	ID(hex)	PID(h...	名称	事件类型	方向	长度	数据(hex)	解释
LIN 1	3C	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
LIN 1	3D	7D		LIN错误	Rx	0		接收错误: 超时, 等待字节 0
LIN 1	3C	3C		LIN帧	Tx	8	11 10 0F 22 FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 21 FF FF FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 22 FF FF FF FF FF FF	
LIN 1	3D	7D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	
LIN 1	3D	7D		LIN错误	Rx	0		接收错误: 超时, 等待字节 0
LIN 1	3C	3C		LIN帧	Tx	8	11 10 0F 22 FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 21 FF FF FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 22 FF FF FF FF FF FF	
LIN 1	3D	7D		LIN帧	Rx	8	11 10 0A 62 FF FF FF FF	
LIN 1	3D	7D		LIN帧	Rx	8	11 21 FF FF FF FF AA FF	

时间轴如下:

产生错误:

- 1) 主机发送 3C_SF, 从机收到后将对应响应写入 TX 队列;
- 2) 主机延时 1.15s, 从机检测到 N_As 超时事件将相关 flag 置为超时状态 (见章节 8.3);
- 3) 主机发出 3D, 由于步骤 2 中从机 tl_service_status 已经置为 LD_SERVICE_ERROR, TX 队列中数据无法发出。

恢复通信:

- 4) 主机发送 3C_FF, 从机收到 3C_FF, 恢复除 tl_tx_msg_status 之外的一切 N_As 相关 flag;
- 5) 主机发送 3C_CF, 从机接收数据;
- 6) 重复步骤 5 直到主从完成最后一帧 3C_CF 的通信, 但因步骤 2 中 tl_tx_msg_status 已经置为 LD_N_AS_TIMEOUT, 从机响应无法写入 TX 队列;
- 7) 主机发出 3D, 从机将步骤 1 中写入 TX 队列中的数据发出;
- 8) 重复步骤 7 直到从机 TX 队列发空, 从机 tl_tx_msg_status 恢复为 LD_COMPLETED;
- 9) 主机发出 3D, 从机无响应, 主机确认从机 TX 队列已空 (此处主机通过多发 3D 指令确保从机 TX 队列清空)。

正常通信:

- 10) 之后主机发送 3C_FF, 3C_CF, 3C_CF, 3D, 3D, 从机均正常收发。

8.5 N_As 超时后通过 SF 帧恢复

N_As 超时事件发生后，可通过 SF 帧恢复正常通信，如图 8-6 所示。

图 8-6: 恢复正常通信

通道	ID(hex)	PID(h...	名称	事件类型	方向	长度	数据(hex)	解释
LIN 1	3C	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
LIN 1	3D	7D		LIN错误	Rx	0		接收错误: 超时, 等待字节 0
LIN 1	3C	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
LIN 1	3D	7D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	

时间轴如下:

产生错误:

- 1) 主机发送 3C_SF, 从机收到后将对应响应写入 TX 队列;
- 2) 主机延时 1.15s, 从机检测到 N_As 超时事件将相关 flag 置为超时状态 (见章节 8.3);
- 3) 主机发出 3D, 由于步骤 2 中从机 tl_service_status 已经置为 LD_SERVICE_ERROR, TX 队列中数据无法发出。

恢复通信 (正常通信):

- 4) 主机发送 3C_SF, 从机收到 3C_SF, 恢复除 tl_tx_msg_status 之外的一切 N_As 相关 flag, 从机响应覆盖 TX 队列中原有数据;
- 5) 主机发出 3D, 从机将 TX 队列中的响应发送出来;
- 6) 重复步骤 7 直到从机 TX 队列发空, 从机 tl_tx_msg_status 恢复为 LD_COMPLETED。

8.6 N_Cr 超时后通过 FF, CF 帧恢复

N_Cr 超时事件发生后, 可通过 FF, CF 帧恢复正常通信, 如图 8-7 所示。

图 8-7: 恢复正常通信

通道	ID(hex)	PID(h...	名称	事件类型	方向	长度	数据(hex)	解释
LIN 1	3C	3C		LIN帧	Tx	8	11 10 0F 22 FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 21 FF FF FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 22 FF FF FF FF FF FF	
LIN 1	3D	7D		LIN错误	Rx	0		接收错误: 超时, 等待字节 0
LIN 1	3D	7D		LIN错误	Rx	0		接收错误: 超时, 等待字节 0
LIN 1	3C	3C		LIN帧	Tx	8	11 10 0F 22 FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 21 FF FF FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 22 FF FF FF FF FF FF	
LIN 1	3D	7D		LIN帧	Rx	8	11 10 0A 62 FF FF FF FF	
LIN 1	3D	7D		LIN帧	Rx	8	11 21 FF FF FF FF AA FF	

时间轴如下:

产生错误:

- 1) 主机发送 3C_FF, 从机收到后写入 RX 队列;
- 2) 主机延时 1.15s, 从机检测到 N_Cr 超时事件将相关 flag 置为超时状态 (见 章节 8.3);
- 3) 主机发送 3C_CF, 由于步骤 2 中从机 tl_service_status 已经置为 LD_SERVICE_ERROR, 从机直接忽略该帧;
- 4) 重复步骤 3 直到完成最后一帧 3C_CF 通信;
- 5) 主机发出 3D, 从机无响应;
- 6) 主机发出 3D, 从机无响应。

恢复通信 (正常通信):

- 7) 主机发出 3C_FF, 从机清空原 RX 队列;
- 8) 主机发送 3C_CF, 从机接收数据;
- 9) 重复步骤 8 直到主从完成最后一帧 3C_CF 的通信, 从机产生响应;
- 10) 主机发出 3D, 从机响应主机;
- 11) 主机发出 3D, 从机响应主机。

8.7 N_Cr 超时后通过 SF 帧恢复

N_Cr 超时事件发生后，可通过 SF 帧恢复正常通信，如图 8-8 所示。

图 8-8: 恢复正常通信

通道	ID(hex)	PID(h...	名称	事件类型	方向	长度	数据(hex)	解释
LIN 1	3C	3C		LIN帧	Tx	8	11 10 0F 22 FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 21 FF FF FF FF FF FF	
LIN 1	3C	3C		LIN帧	Tx	8	11 22 FF FF FF FF FF FF	
LIN 1	3D	7D		LIN错误	Rx	0		接收错误：超时，等待字节 0
LIN 1	3D	7D		LIN错误	Rx	0		接收错误：超时，等待字节 0
LIN 1	3C	3C		LIN帧	Tx	8	11 06 B2 00 21 00 01 00	
LIN 1	3D	7D		LIN帧	Rx	8	11 06 F2 21 00 01 00 00	

时间轴如下：

产生错误：

- 1) 主机发送 3C_FF，从机收到后写入 RX 队列；
- 2) 主机延时 1.15s，从机检测到 N_Cr 超时事件将相关 flag 置为超时状态（见章节 8.3）；
- 3) 主机发送 3C_CF，由于步骤 2 中从机 tl_service_status 已经置为 LD_SERVICE_ERROR，从机直接忽略该帧；
- 4) 重复步骤 3 直到完成最后一帧 3C_CF 通信；
- 5) 主机发出 3D，从机无响应；
- 6) 主机发出 3D，从机无响应。

恢复通信（正常通信）：

- 7) 主机发出 3C_SF，从机清空原 RX 队列，从机产生响应；
- 8) 主机发出 3D，从机响应主机。